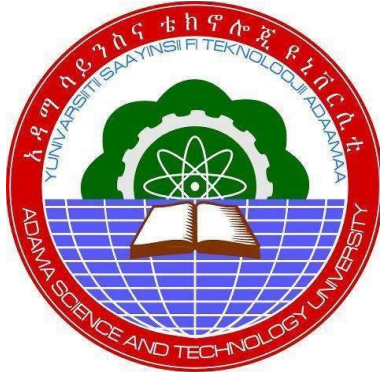




ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTING

DEPARTMENT OF SOFTWARE ENGINEERING

Course Title: Component Based Software Development

Course Code: SEng5306

Group Project: Monorepo System Development

1, Dagmawi Adane	UGR/26689/14
2, Fitsum Teka	UGR/25652/14
3, Yohanan Solomon	UGR/25476/14
4, Meriat Akalu	UGR/25909/14
5, Hanamariam Mesfin	UGR/25483/14

Submitted to: MR. Daniel Metiku

1. Executive Summary

The **BizFlow Enterprise System** is a **modular enterprise platform** engineered using a **strict monorepo architecture**. The system demonstrates how large-scale applications can be decomposed into **independent, reusable, and composable feature modules**.

Instead of building a single monolithic system, BizFlow is split into:

- **Two independent applications**
- **Ten isolated feature packages**
- **Two shared foundational libraries**

Core Architectural Philosophy

Applications assemble — packages implement.

- Applications contain **zero business logic**
- Features encapsulate **all domain logic**
- Shared packages provide **pure reusable primitives**

2. Architectural Vision

BizFlow is designed around **three fundamental principles**:

1. Separation of Concerns

- UI, logic, and orchestration are split across layers

2. Reusability over Duplication

- Every feature is written once and reused everywhere

3. Dependency Control

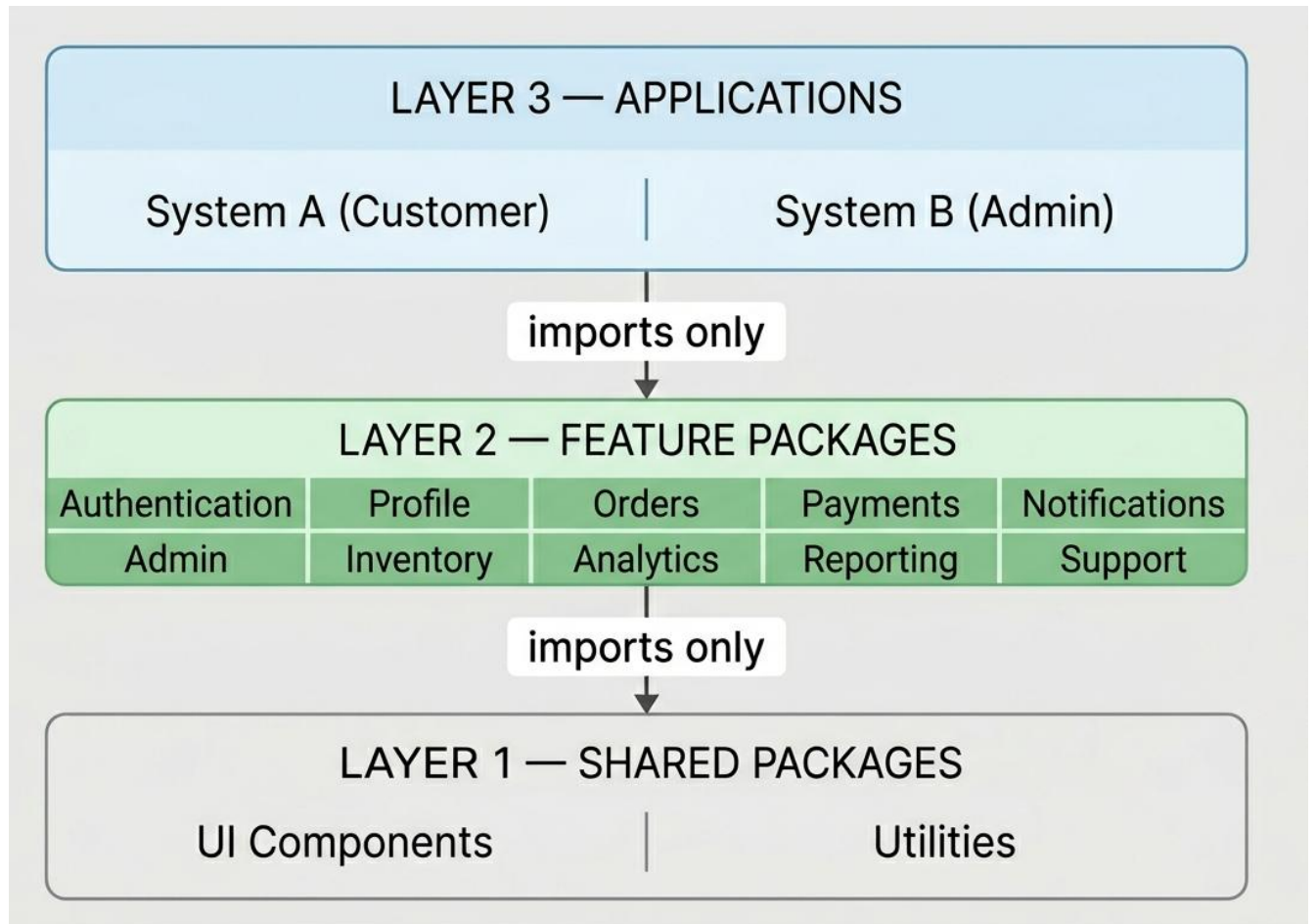
- Strict one-directional dependency flow prevents coupling

3. Group Members and Task Assignment

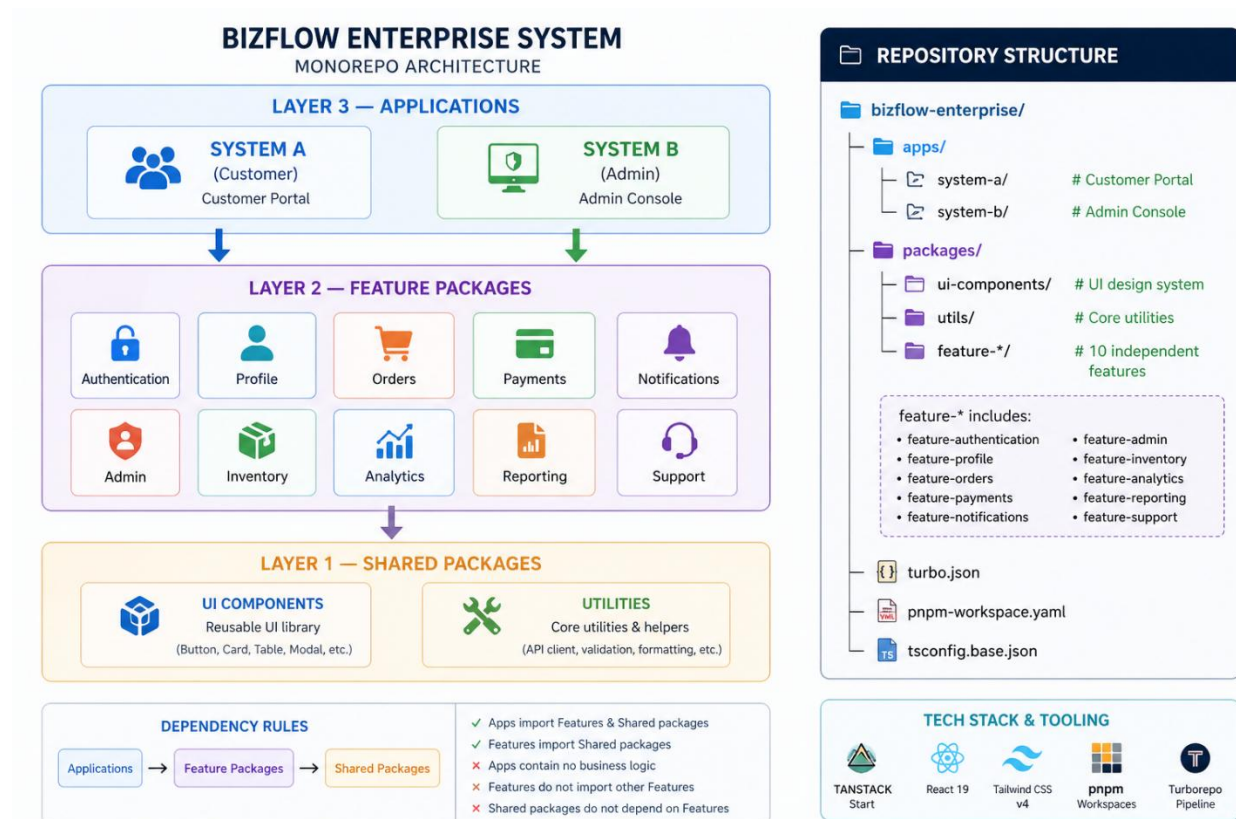
#	Full Name	Individual Features	Group Contribution
1	Dagmawi Adane	Inventory, Analytics	Monorepo setup, base UI components
2	Fitsum Teka	Orders, Payments	UI components (Table, Modal), System A
3	Yohanan Solomon	Auth, Profile	Utilities, System B
4	Meriat Akalu	Notifications, Support	Layout, design tokens
5	Hanamariam Mesfin	Admin, Reporting	Documentation, QA, utilities

4. High-Level Architecture

Three-Layer System Design



5. Repository Structure



6. Layer 1 — Shared Packages

6.1 UI Components (@bizflow/ui-components)

A **design system layer** providing reusable UI primitives.

Key properties:

- Stateless where possible
- No business logic
- Fully reusable across all features

Example Components:

- Button, Input, Card
- Table, Modal, Badge
- StatCard, PageHeader
- AppShell (layout system)

Design System:

- Built with Tailwind v4
- Uses **OKLCH color tokens**
- Supports dark/light modes

6.2 Utilities (@bizflow/utils)

A **pure logic layer** with no UI dependencies.

Responsibilities:

- API abstraction
- Data formatting
- Validation
- Aggregation

Examples:

- `apiClient()` → centralized HTTP logic
- `formatCurrency()` → consistent formatting
- `validateEmail()` → validation logic
- `aggregate()` → analytics calculations

7. Layer 2 — Feature Packages

Each feature is a **self-contained domain module**.

Feature Characteristics

- Owns its state
- Owns its API calls
- Owns its UI composition
- Exports public components

Feature List

Feature	Responsibility
Auth	Authentication & sessions
Profile	User account management
Orders	Order lifecycle
Payments	Payment processing

Notifications	Alerts system
Inventory	Stock management
Analytics	KPIs & dashboards
Support	Ticketing system
Admin	User roles & config
Reporting	Reports & exports

8. Layer 3 — Applications

Applications are **pure composition layers**.

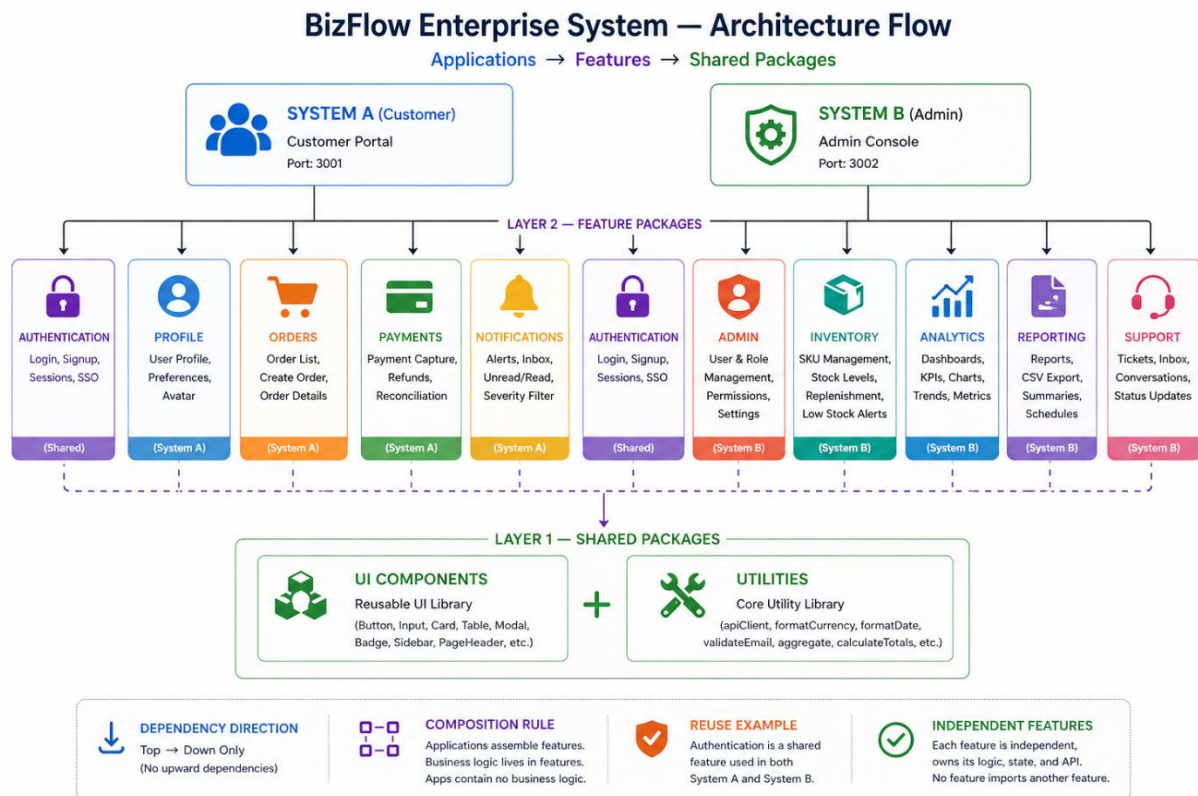
System A — Customer Portal

- Auth
- Profile
- Orders
- Payments
- Notifications

System B — Admin Console

- Auth (reused)
- Inventory
- Analytics
- Support
- Admin
- Reporting

9. Dependency Graph



10. Data Flow Architecture

Read Flow

User → App Route → Feature Component → Hook → API Client → Backend

→ Utils (formatting) → UI Components → Render

Write Flow

User Action → Feature Component → Validation (utils)

→ API Client → Backend

→ UI Feedback (shared components)

11. Architectural Advantages

1. True Reusability

- feature-auth reused across both systems without modification

2. Independent Development

- Each developer works on isolated packages

3. Scalability

- New features = new packages (no refactor needed)

4. Maintainability

- Bug fix in one place propagates everywhere

5. Performance

- Turborepo caching reduces build time

12. Anti-Patterns Prevented

This architecture explicitly prevents:

- ❌ Business logic inside apps
- ❌ Tight coupling between features
- ❌ Duplicate implementations

-  Uncontrolled dependencies

13. Quality Assurance Strategy

Layer	Testing Focus
utils	Pure logic correctness
UI	Rendering & accessibility
Features	Behavior & API interactions
Apps	Integration (routing only)

14. Key Architectural Decisions

Decision	Impact
Monorepo over polyrepo	Faster development & integration
Strict layer separation	Clean architecture
Feature isolation	High scalability
Shared design system	UI consistency
Central utilities	Logic reuse

15. Conclusion

The **BizFlow Enterprise System** successfully demonstrates a **production-grade monorepo architecture** with:

- Clear separation of layers
- Strong dependency control
- High code reusability
- Scalable feature-based design

Final Insight

BizFlow is not just an application — it is a **system architecture blueprint** for building scalable enterprise software.

16. References

- TanStack Start
- Turborepo Documentation
- pnpm Workspaces
- Tailwind CSS v4
- ShadCN UI Patterns
- Monorepo Architecture Guides